Section 2: Prerequisite Representation

Modeling complex AND/OR logic for automated validation

The Core Problem

Prerequisites aren't simple lists. They have **logic**:

> **SYSC 4001:**
> "SYSC 3303 **and** (SYSC 3310 **or** SYSC 3320)"
>
> **SYSC 4805:**
> "Fourth-year standing **or** permission"

We need to:

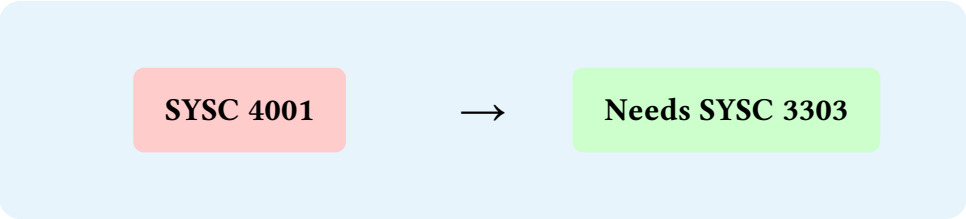
- Store this logic in a database
- Evaluate it against a student's transcript
- Show clear feedback (pass/fail + what's missing)

Requirements

1. **Persist in relational DB** — can't just store strings
2. **Evaluate programmatically** — no human interpretation
3. **Support multiple node types:**
   - Course requirements (SYSC3303)
   - Year standing (Fourth-year)
   - Special permission
   - Grade requirements (C- minimum)
4. **Handle nested logic** — AND inside OR inside AND
5. **Provide actionable feedback** — list missing courses

What Are Prerequisites?

A prerequisite is a rule: “Before you take X, you must first take Y”



But sometimes the rules get more complex…

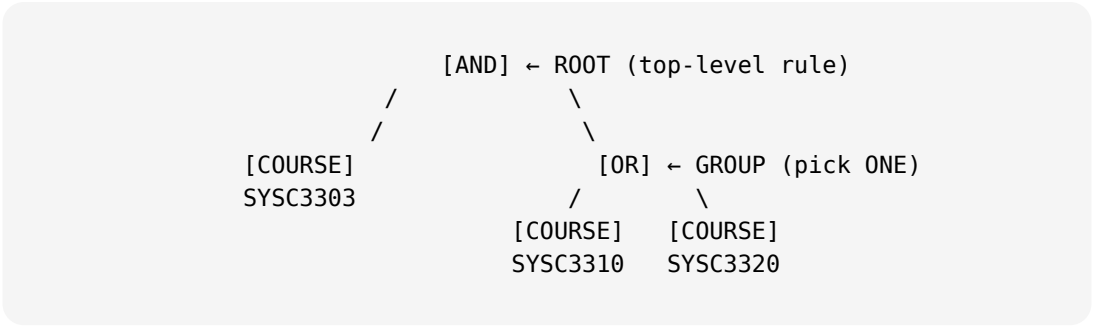
SYSC 4001 requires:
“SYSC 3303 AND (SYSC 3310 OR SYSC 3320)”

This means you need:

- SYSC 3303 ✓
- AND at least one of: SYSC 3310 OR SYSC 3320 ✓

The Tree Structure

Think of prerequisites like a family tree



AND node:
Everything under it must be true

, [

OR node:
You only need ONE of the options

,]

💡 **Key insight:** Complex rules become simple when drawn as trees!

Prerequisite Schema: Database Design

Self-referential tree structure in PostgreSQL via Prisma

PrerequisiteNode Model

```
model PrerequisiteNode {
  id               String   @id @default(cuid())
  courseId         String   // Which course this
prereq belongs to
  nodeType         String   // AND | OR | COURSE |
YEAR_STANDING | PERMISSION

  // For COURSE nodes
  requiredCourseId String?  // FK to Course
  requiredGrade    String?  // e.g., "C-"

  // For YEAR_STANDING nodes
  requiredYear     String?  // FIRST_YEAR |
SECOND_YEAR | etc.

  // Tree structure (self-reference)
  parentId         String?
  parent           PrerequisiteNode?
@relation("NodeHierarchy")
  children         PrerequisiteNode[]
@relation("NodeHierarchy")

  // Relations
  course           Course
@relation("CoursePrerequisites")
  requiredCourse   Course?
@relation("RequiredByCourses")
}
```

Why Self-Reference?

- Trees need parent-child relationships
- `parentId` points to another node in same table
- Root nodes have `parentId = null`
- Prisma handles the FK automatically

Node Types

Type	Meaning
AND	All children must pass
OR	Any child must pass
COURSE	Specific course required
YEAR_STANDING	Minimum year level
PERMISSION	Manual override needed

How We Store the Tree

Each row in the table represents one “box” in the tree

id	nodeType	requiredCourse	parentId	Meaning
node1	AND	null	null	ROOT - must pass everything
node2	COURSE	SYSC3303	node1	Need SYSC 3303
node3	OR	null	node1	Need one of these
node4	COURSE	SYSC3310	node3	Option A
node5	COURSE	SYSC3320	node3	Option B

Left side: The database table rows
Right side: What each row represents in the tree

Prerequisite Parser: Expression → Tree

Converting human-readable expressions to node trees

Input Format (from course calendar)

```
"SYSC3303 && (SYSC3310 || SYSC3320)"
"FOURTH_YEAR || PERMISSION"
"COMP1006[C-] && MATH1004"
```

Parser Strategy

1. Split on && first (lower precedence)
2. Split on || within groups
3. Handle parentheses for nesting
4. Parse individual terms (courses, years, etc.)

Parsing Logic

```
function parsePrerequisiteExpression(expr:
string): ParsedNode {
  // Handle simple AND (no OR present)
  if (expr.includes(' && ') && !expr.includes('
|| ')) {
    const parts = expr.split(' && ')
    return {
      type: 'AND',
      children: parts.map(parseSimpleTerm)
    }
  }

  // Handle simple OR (no AND present)
  if (expr.includes(' || ') && !expr.includes('
&& ')) {
    const parts = expr.split(' || ')
    return {
      type: 'OR',
      children: parts.map(parseSimpleTerm)
    }
  }

  // Handle parentheses - recursive
  if (expr.includes('(')) {
    return parseComplexExpression(expr)
  }

  return parseSimpleTerm(expr)
}
```

From Text to Tree: Step by Step

How the parser reads: “SYSC3303 && (SYSC3310 || SYSC3320)”

Step 1: Find AND

```
"SYSC3303" && "(SYSC3310
    || SYSC3320)"
```

Two parts separated by AND

, [

Step 2: Find OR in parentheses

```
"SYSC3310" || "SYSC3320"
```

OR group inside ( )

], [

Step 3: Build the tree

```
        [AND]
       /     \
  [COURSE]    [OR]
  SYSC3303   /    \
           [...]  [...]
```

]

Result: A tree structure the computer can evaluate!

Prerequisite Parser: Term Recognition

Identifying different types of requirements

parseSimpleTerm Function

```
function parseSimpleTerm(term: string):
ParsedNode {
  term = term.trim().replace(/[()]/g, '')

  // Year standing
  if (['FIRST_YEAR', 'SECOND_YEAR',
       'THIRD_YEAR',
'FOURTH_YEAR'].includes(term)) {
    return { type: 'YEAR_STANDING', value:
term }
  }

  // Permission
  if (term === 'PERMISSION') {
    return { type: 'PERMISSION' }
  }

  // Course with grade requirement
  const gradeMatch = term.match(
    /^([A-Z]{3,4}\d{4})\[([A-Z\-+]*)\]$/
  )
  if (gradeMatch) {
    return {
      type: 'COURSE',
      value: gradeMatch[1],  // COMP1006
      grade: gradeMatch[2]   // C-
    }
  }

  // Standard course
  if (term.match(/^[A-Z]{3,4}\d{4}$/)) {
    return { type: 'COURSE', value: term }
  }

  // Unknown - treat as permission
  return { type: 'PERMISSION' }
}
```

Example Transformations

Input	Output Node
"SYSC3303"	{type: 'COURSE', value: 'SYSC3303'}
"COMP1006[C-]"	{type: 'COURSE', value: 'COMP1006', grade: 'C-'}
"FOURTH_YEAR"	{type: 'YEAR_STANDING', value: 'FOURTH_YEAR'}
"PERMISSION"	{type: 'PERMISSION'}

Fallback behavior: Unknown terms become PERMISSION nodes to avoid foreign key errors (logged as warnings).

Prerequisite Storage: Building the Tree

Recursively creating nodes in PostgreSQL

Recursive Tree Creation

```
async function createPrerequisiteTree(
  courseId: string,
  parsedNode: ParsedNode,
  parentId?: string
): Promise<string> {

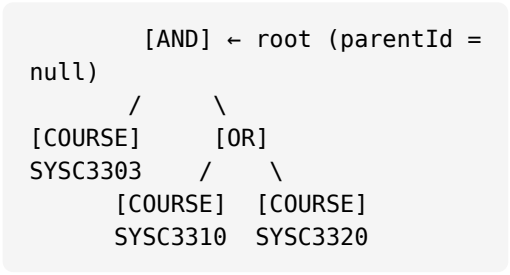
  // Create this node
  const node = await
prisma.prerequisiteNode.create({
    data: {
      courseId,
      nodeType: parsedNode.type,
      requiredCourseId: parsedNode.type ===
'COURSE'
        ? parsedNode.value : null,
      requiredYear: parsedNode.type ===
'YEAR_STANDING'
        ? parsedNode.value : null,
      requiredGrade: parsedNode.grade || null,
      parentId: parentId || null
    }
  })

  // Recursively create children
  if (parsedNode.children) {
    for (const child of parsedNode.children) {
      await createPrerequisiteTree(
        courseId, child, node.id
      )
    }
  }

  return node.id
}
```

Visual Example

Expression: `"SYSC3303 && (SYSC3310 || SYSC3320)"`



Each box = one row in `prerequisite_nodes` table
Links via `parentId` foreign key

Prerequisite Validation: Evaluation Algorithm

Recursive tree traversal with boolean logic

Core Evaluation Function

```
async function evaluatePrerequisiteNode(
  node: PrerequisiteNodeData,
  transcript: StudentTranscript,
  allNodes: PrerequisiteNodeData[]
): Promise<EvaluationResult> {

  switch (node.nodeType) {
    case 'AND': {
      const children = allNodes.filter(
        n => n.parentId === node.id
      )
      const results = await Promise.all(
        children.map(c =>
          evaluatePrerequisiteNode(c,
transcript, allNodes)
        )
      )
      const failed = results.filter(r => !
r.isValid)
      return {
        isValid: failed.length === 0,
        missingItems: failed.flatMap(r =>
r.missingItems)
      }
    }

    case 'OR': {
      // Similar but returns true if ANY child
passes
    }

    case 'COURSE': {
      // Check if course in transcript
    }
  }
}
```

Node Type Handlers

Type	Logic
AND	All children must return `isValid: true`
OR	At least one child must return `isValid: true`
COURSE	Check `transcript.completedCourses.includes(cours`
YEAR	Compare `transcript.currentYear >= requiredYear`
PERM	Check `transcript.hasSpecialPermission`

Complexity Analysis

- **Time:** O(n) where n = number of nodes
- **Space:** O(h) where h = tree height (recursion stack)
- Typical prerequisite tree: 5-15 nodes

How the System Checks Prerequisites

Walking through the tree with a student's transcript

Student's Transcript: [], • SYSC 3303 (completed), • SYSC 2006 (completed), • MATH 1004 (completed), • **Missing: SYSC 3310, SYSC 3320**,

```
          [AND] ← Check: All children must pass
         /     \
[✓ SYSC3303]  [OR] ← Check: Any ONE child must pass
             /    \
        [✗ SYSC3310] [✗ SYSC3320]

        RESULT: FAILED (OR group didn't pass)
```

Feedback to student: "You need SYSC 3303 AND (SYSC 3310 OR SYSC 3320)
You have SYSC 3303 ✓
You're missing: SYSC 3310 OR SYSC 3320"

Prerequisite Validation: Grade Requirements

Some courses require a minimum grade, not just completion

Grade Scale Implementation

```
function meetsGradeRequirement(
  actualGrade: string | undefined,
  requiredGrade: string
): boolean {
  if (!actualGrade) return false

  const gradeValues: Record<string, number> = {
    'A+': 12, 'A': 11, 'A-': 10,
    'B+': 9,  'B': 8,  'B-': 7,
    'C+': 6,  'C': 5,  'C-': 4,
    'D+': 3,  'D': 2,  'D-': 1,
    'F': 0
  }

  const actualValue = gradeValues[actualGrade]
  const requiredValue =
gradeValues[requiredGrade]

  // Unknown grades: be permissive
  if (actualValue === undefined ||
      requiredValue === undefined) {
    return true
  }

  return actualValue >= requiredValue
}
```

Usage in COURSE Node Evaluation

```
case 'COURSE': {
  const hasCourse =
transcript.completedCourses
    .includes(node.requiredCourseId)

  if (!hasCourse) {
    return {
      isValid: false,
      reason: `Missing
${node.requiredCourseId}`,
      missingItems: [node.requiredCourseId]
    }
  }

  // Check grade if required
  if (node.requiredGrade) {
    const actual =
transcript.courseGrades?.[
      node.requiredCourseId
    ]
    if (!meetsGradeRequirement(actual,
node.requiredGrade)) {
      return {
        isValid: false,
        gradeIssues:
[`${node.requiredCourseId}: need
${node.requiredGrade}, have ${actual}`]
      }
    }
  }

  return { isValid: true }
}
```

Prerequisite Validation: User Feedback

Providing actionable information, not just pass/fail

ValidationResult Interface

```
interface PrerequisiteValidationResult {
  isValid: boolean
  reason?: string
  missingCourses?: string[]    // ["SYSC3303",
"SYSC3310"]
  yearRequirement?: string     // "FOURTH_YEAR"
  needsPermission?: boolean
  gradeIssues?: string[]       // ["COMP1006:
need C-, have D+"]
  expressionEvaluated: string  // Original
expression
}
```

We return **all** missing items, not just the first one found. This helps students plan which courses to take.

Example Output

Course: SYSC 4001
Status: FAIL
Required: `SYSC3303 && (SYSC3310 || SYSC3320)`
Missing courses: SYSC3303
Details: You need SYSC3303 AND at least one of SYSC3310/SYSC3320

Course: SYSC 4907
Status: PASS
Required: `FOURTH_YEAR`
Details: You meet all prerequisites

Before vs After: Student Experience

What students see with our system vs. the old way

OLD WAY

1. Look up course on calendar
2. Read prerequisite text
3. Try to register
4. **Error:** “Prerequisites not met”

But WHICH prerequisite?
What grade do I need?
What year standing?

, [

NEW WAY

1. Select course
2. System checks automatically

Result: ✅ PASS
or
❌ FAIL: “Missing SYSC 3310 or SYSC 3320”

Clear, specific feedback
Actionable next steps

]

Bottom line: Students know exactly what they need to do next, instead of guessing.

Section 2: Summary

Key concepts and what they enable

What We Built

• Tree-based data model for prerequisites
• Parser to convert text expressions to trees
• Recursive evaluation algorithm
• Grade requirement support
• Clear user feedback system , [**Why It Matters**

• No more guessing about requirements
• Automatic validation saves time
• Supports complex AND/OR logic
• Works for any course in the catalog
• Foundation for schedule generation ,]

Up next: Section 3 – How we use this to generate conflict-free schedules